

Location is Key: Leveraging LLM for Functional Bug Localization in Verilog Design

Bingkun Yao
City University of Hong Kong
bingkun.yao@cityu.edu.hk

Ning Wang
City University of Hong Kong
nwang227-c@my.cityu.edu.hk

Jie Zhou
Southeast University
zhoujie_0428@163.com

Xi Wang
Southeast University
xi.wang@seu.edu.cn

Hong Gao
Zhejiang Normal University
honggao@zjnu.edu.cn

Zhe Jiang*
Southeast University
101013615@seu.edu.cn

Nan Guan*
City University of Hong Kong
nanguan@cityu.edu.hk

Abstract—In Verilog code design, identifying and locating functional bugs is an important yet challenging task. Existing automatic bug localization methods have limited capabilities; they only suggest a set of potential buggy lines rather than precisely identifying the bug. Moreover, they depend on verification tools like testbenches and reference models, which require expert input and are time-consuming to develop. This paper introduces LiK (Location is Key), an open-source Large Language Model (LLM) to precisely locate functional bugs in Verilog code without the need for expert-written verification tools. LiK is developed from the open-source coding LLM Deepseek-Coder-Lite-Base-16B through a three-step training process: continuous pre-training to enhance foundational knowledge, supervised fine-tuning to learn how to output localization results, and reinforcement learning to reduce output errors. Experiment results demonstrate that LiK achieves superior functional bug localization accuracy, outperforming both the SOTA traditional method Strider, and SOTA closed-source LLMs like GPT-o1-preview and Claude-3.5-Sonnet. Moreover, integrating LiK into the SOTA LLM-based Verilog debugging tool significantly boosts its functional bug fixing success rate from 76.47% to 90.54%. This underscores LiK’s potential to enhance the performance of end-to-end automatic Verilog debugging tools.

I. INTRODUCTION

To fix bugs in Verilog code design, the first step is to identify and locate them within the code. Unlike syntax bugs, functional bugs can pass syntax checks and synthesis but cause the circuit to behave unexpectedly, making them much more challenging to detect and pinpoint. Therefore, an effective tool that automatically locates functional bugs would be immensely beneficial to Verilog developers, significantly enhancing their debugging efficiency.

Existing methods [1]–[9] for automatically locating functional bugs in Verilog code have significant limitations. They typically suggest a set of potential buggy lines rather than pinpointing the exact line containing the bug. Additionally, these methods rely on verification tools like testbenches and reference models, which require significant expertise and are time-consuming to develop. These limitations reduce the practical value of existing automated bug localization techniques for Verilog developers.

In this paper, we propose a novel approach to automate Verilog functional bug localization using Large Language Models (LLMs). Our method aims to identify buggy lines precisely by analyzing the code directly, eliminating the need for expert-written verification tools or Electronic Design Automation (EDA) tools. Over the past two years, researchers have extensively investigated the application of LLMs in software debugging [10]–[12]. By leveraging the powerful code comprehension capabilities of LLMs, these approaches have demonstrated remarkable effectiveness in automated bug detection and repair. These promising results motivate us to explore LLM-based approaches for Verilog functional bug localization, an area that remains unexplored in current research.

We introduce LiK (Location is Key), an open-source LLM for *functional* bug localization in Verilog code. LiK is devel-

* Corresponding authors.

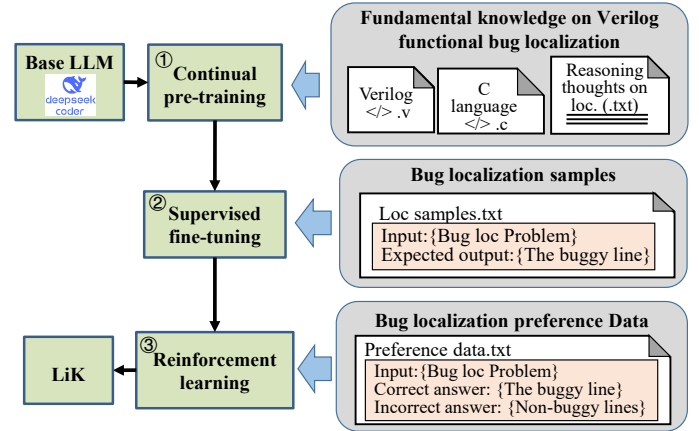


Fig. 1. The training process of LiK, including three steps: ① Unsupervised continuous pre-training to enhance the foundational knowledge in Verilog bug localization; ② Supervised fine-tuning to enable the LLM to output the correct bug location; ③ Reinforcement learning to reduce common errors in localization results.

oped from Deepseek-Coder-Lite-Base-16B [13], a widely-used open-source LLM specialized for coding tasks. Our approach includes three steps: (1) continuous pre-training to enhance LiK’s foundational knowledge in Verilog bug localization; (2) supervised fine-tuning to teach LiK to output localization results and (3) reinforcement learning [14] to reduce common localization errors. Our experiments show that LiK achieves superior localization accuracy, surpassing both the SOTA non-LLM method Strider [9] and leading closed-source LLMs including GPT-o1-preview and Claude-3.5-Sonnet.

LiK serves two practical applications: it functions as an IDE add-on to assist developers, and as an enhancement module for auto-verification tools to support debugging. We incorporate LiK into GPT-3.5, GPT-4, and the SOTA LLM-based Verilog debugging system MEIC [15], which significantly improves their bug fixing performance. In particular, MEIC’s success rate on functional bug fixing increases from 76.47% to 90.54%. This result demonstrates LiK’s potential in strengthening the performance of automatic Verilog debugging systems. Datasets and code of this research will be open-sourced at <https://github.com/bingkunyao/Location-is-key>.

II. OUR METHOD

We develop LiK from Deepseek-coder-V2-Lite-base-16B [13] (referred to as the base LLM), an open-source LLM that excels in coding tasks. As shown in Fig. 1, we obtain LiK from the base LLM through three training steps:

- 1) *Continual pre-training*. Since the base LLM lacks specialized knowledge in Verilog bug localization, we perform two rounds of

unsupervised continual pre-training to enhance its fundamental knowledge in this domain.

- 2) *Supervised fine-tuning.* We fine-tune the model under the supervision of bug localization samples to enable it to generate bug localization results.
- 3) *Reinforcement learning.* While supervised fine-tuning teaches the model “what is correct”, we leverage reinforcement learning [14] to help it learn “what to avoid” from both correct and incorrect answers, thereby reducing common localization errors.

A. Continual Pre-training

We perform two rounds of unsupervised continual pre-training on the base LLM. The first round enhances the model’s fundamental knowledge of Verilog, while the second round focuses on knowledge specific to functional bug localization.

We collect Verilog data for training from three sources: (a) VGen [16] dataset which includes all open-source Verilog repositories from GitHub; (b) OpenCores [17], which is a major open-source hardware design platform, and (c) an internal code dataset acquired from our partnered hardware design companies. Even with our exhaustive data collection efforts, the total data size remains limited (~253MB). This stems from the closed nature of the hardware design industry, where most Verilog designs are private and inaccessible. Given this data scarcity, we employ the LoRA [18] algorithm for continual pre-training, which updates only a small subset of all the model parameters. This targeted parameter updating approach effectively mitigates the risk of overfitting when conducting continual pre-training with limited data.

1) Round 1: For Basic Knowledge of Verilog

We pre-train the base LLM with fundamental knowledge pertaining to Verilog. Notably, our training corpus includes not only Verilog but also C language. This is motivated by the significant syntactical similarities between them [19]. Although the base LLM already contains C language knowledge from its initial training, additional C language exposure during the pre-training phase can further enhance the model’s understanding of these shared syntactical features [20], leading to better comprehension of Verilog.

The Verilog corpus comes from the VGen [16] dataset and consists of two parts. The first part comprises open-source Verilog files from GitHub, where each file contains “module” and “endmodule” statements and has fewer than 20000 characters. The second part consists of sections from 70 Verilog textbooks in e-PDF format, where sections without Verilog codes are filtered out.

For the C language corpus, we use the documentation from the CodeSearchNet [21] dataset. The execution of C is fundamentally different from that of Verilog: C language executes sequentially, while Verilog describes hardware circuits where multiple operations can occur simultaneously. Therefore, we limit the C language content to less than 20% of the total corpus. This helps maintain the LLM’s focus on Verilog’s hardware description features.

We apply MinHash [22] and LSH [23] algorithms for data deduplication and remove any syntactically incorrect Verilog and C code. Through these preprocessing steps, we refine the final corpus to 91787 entries, where each entry is a code snippet or documentation of Verilog/C language. The size of the final corpus is 206.48MB, with Verilog content comprising 171.03MB and C language materials accounting for 35.45MB.

2) Round 2: For Knowledge Specific to Bug Localization

After enhancing the model’s fundamental Verilog knowledge, our next goal is to incorporate specialized knowledge for bug localization.

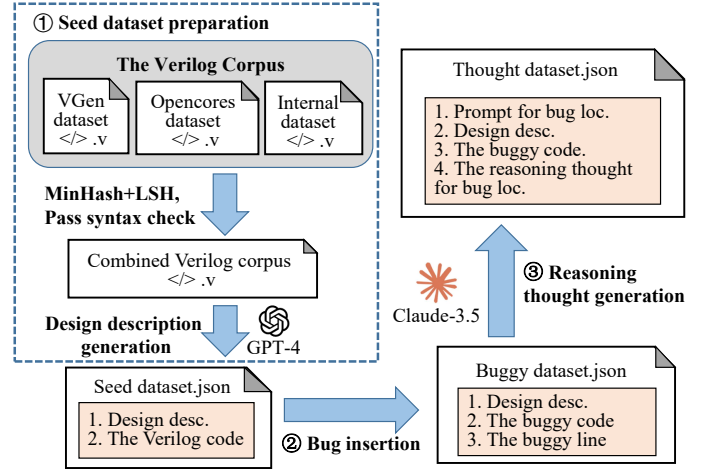


Fig. 2. The process of synthesize the training data including the reasoning thoughts for bug localization.

Research [24] demonstrates that integrating reasoning processes that lead to correct answers into pre-training helps the models better understand the input tasks and generate more relevant responses. Following this principle, we use reasoning thoughts for bug localization as our pre-training corpus.

We formulate each training data as:

$$f \circ d \circ b \circ r, \quad (1)$$

where $\circ$ represents the concatenate operator, f denotes a prompt that instructs the model to locate the buggy line, d represents the design description of the code, b denotes the buggy code, and r denotes the reasoning thought that explains how to locate bugs in b . Fig. 3 illustrates an example of this training data. Since such data is scarce in open-source communities, we develop a synthetic data generation approach, which is shown in Fig. 2: First, we prepare a seed dataset containing paired design descriptions and code. Next, we insert bugs into the code. Finally, we employ Claude-3.5-Sonnet to generate reasoning thoughts for bug localization.

(1) **Seed dataset.** Our seed dataset integrates Verilog code from three sources: VGen [16], OpenCores [17], and an internal code dataset acquired from our partnered hardware design companies. Each dataset entry is a Verilog module, which serves as the basic units for our training data. We remove duplicate modules by the MinHash [22] and LSH [23] algorithms, and include modules that pass syntax checks and contain more than 20 lines of code. For each module, we use GPT-4 to generate the corresponding design description. The final seed dataset comprises 7802 entries, totaling 36.92MB.

(2) **Bug insertion.** Based on our analysis of [25], we insert bugs according to five common functional error patterns in Verilog:

- *Operator errors:* Misuse of operators, e.g., confusing “+” with “,”; “&&” with “|”.
- *Numerical value errors:* Incorrect numerical values or bit widths, e.g., setting a clock divider ratio to 8 instead of the required 16, declaring a 4-bit signal when 6 bits are required.
- *Keyword errors:* Incorrect use of complementary keywords, e.g., confusing “wire” with “reg”, “input” with “output”.
- *Variable name errors:* Incorrect variable references, e.g., confusing “counter1” with “counter2”.
- *Edge errors:* Incorrect specifications on edge conditions, e.g., confusing a rising edge “posedge” with a falling edge “negedge”.

For each data entry in the seed dataset, we select one of the above patterns and use regular expressions to modify a random location in the code, thus obtaining the buggy code and buggy line. Since

The prompt of bug localization f :
 # You are a hardware engineer proficient in Verilog. Please locate the functional bug in the code according to the design description and the code.

Design description d :
 // Provide the input/output interfaces and functional requirements
 # Module name: ...
 # Interface Description: Input ports: ... Output ports: ...
 # Functional requirements: ...

The buggy code b :
 # {The buggy code}

The reasoning thought for bug localization r :
 # Let's analyze the code step-by-step to locate the buggy line.
 // Analyze the code from the front to the end
 # 1. Module Interface: It seems that there is an error with the output port. According to the design description, ...
 # 2. Internal Registers: The internal registers matches well with design descriptions, ...
 ...
 # After careful analysis, we conclude that the buggy line is ``output [16 + 46 - 1:0] C;`` // Provide the buggy line at the end of reasoning thought

Fig. 3. A training sample for the second round of continual pre-training.

we focus on functional bugs, we retain only those entries where the buggy code is syntactically correct.

(3) **Reasoning thought generation.** For each data entry, we construct a prompt containing the design description d , the buggy code b , and the buggy line, then feed it to Claude-3.5-Sonnet to generate reasoning thoughts that pinpoint the buggy line in b . Our prompt instructs the model to follow a step-by-step reasoning process, assuming the buggy line is unknown at the beginning of the reasoning process. This prevents the model from directly stating the buggy line at the beginning of its reasoning, which aligns with real-world scenarios where developers do not know the bug location when starting to debug. Additionally, some generated thoughts fail to locate the actual buggy line, and manually checking each reasoning thought would be labor-intensive. Therefore, we require the model to conclude the reasoning thought with the specific format “the buggy line is {...}.”. This formal specification allows us to automatically verify whether the reasoning leads to the correct buggy line.

We randomly select 100 generated thoughts for manual review. We observe that each thought follows a systematic reasoning process similar to human debugging. The reasoning process examines the buggy code from front to end and ultimately pinpoints the buggy line. Fig. 3 illustrates an example of these generated thoughts. After removing thoughts that fail to locate the actual buggy line, we obtain a final dataset of 7622 entries, totaling 46.83MB in size.

B. Supervised Fine-tuning

The base LLM acquires essential knowledge for functional bug localization through continual pre-training. However, due to the unsupervised nature of pre-training, the base LLM cannot directly answer the question “Where is the bug?”. To address this issue, we fine-tune the model under the supervision of bug localization samples. Same as continual pre-training, since our training data is quite limited (36.72MB), we employ the LoRA algorithm for supervised fine-tuning to prevent overfitting.

Each training sample is an input-output pair (q, a) . The input q is a natural language description of the bug localization problem, including both the design description and the buggy code. For the expected output a , which indicates the bug location, there are two possible formats: the line number (as in existing non-LLM solutions) or the content of the actual buggy line. We choose the latter because LLMs are better at processing natural language text than numerical reasoning, as these are trained primarily using textual data. Fig. 4 illustrates an example of such a training sample.

We apply the same seed dataset and method as in Sec. II-A2 to generate the training data for supervised fine-tuning, producing 7622 data samples with a total size of 36.72MB.

Input q :
 # You are a hardware engineer proficient in Verilog. Below I give you a hardware design description and the corresponding code implementation with the buggy line. Your task is to identify this buggy line.
 # Hint: the buggy line may not conform to the design description, or has logic/functional error.
 # Requirements: You should only output the SPECIFIC buggy line identified.
 // The output is the specific content of the buggy line
 # Design description: {Design description}
 # Code implementation: {Buggy code}

Expected output a :
 # always @ (negedge start) begin //The content of the buggy line

Fig. 4. A training sample for supervised fine-tuning.

While recent literature [24] suggests that prompting LLMs to explain their reasoning before providing answers can improve accuracy, we only use the buggy line content as the expected output of the model, without including the reasoning thoughts for bug localization. This is because reasoning thoughts may introduce noise during the supervised fine-tuning, leading to a decrease in localization accuracy. We discuss this issue in detail in Sec. IV-B.

C. Reinforcement Learning

Supervised fine-tuning trains the model using correct bug localizations but does not actively discourage incorrect ones. To overcome this limitation, we employ reinforcement learning (RL) techniques [14] to let the model learn from both correct and incorrect answers. This helps avoid common localization errors, thereby further enhancing the model’s overall performance.

We implement SimPO algorithm [26] for RL training. SimPO directly aligns the reward with the model’s generation process. This alignment encourages outputs matching correct answers while discouraging those that resemble incorrect ones.

During training, SimPO optimizes model parameters through gradient descent by maximizing an objective function. This function measures the reward difference between correct and incorrect answers, thereby enhancing the alignment between model outputs and correct bug locations. The following sections describe our training data, reward function, and objective function in detail.

1) Training Data: Correct and Incorrect Answers

Each training sample is a triple (q, a_w, a_l) , where q represents a bug localization problem, a_w denotes the correct answer (i.e., the actual buggy line), and a_l represents the incorrect answer, which is a non-buggy line that the model should avoid. To construct these training samples, we first generate (q, a_w) pairs using the approach and seed dataset described in Sec. II-A2. To identify a_l for each pair, we repeatedly feed q to the model, then select the non-buggy line that the model misidentifies most frequently. Fig. 5 illustrates an example of such a training sample.

We initially generate 7622 (q, a_w) pairs. For 4080 of these pairs, the model consistently returns the actual buggy line, making it impossible to obtain a suitable a_l . After removing these pairs, our final dataset includes 3542 samples with a total size of 19.82MB.

2) The Reward and Objective Functions of SimPO

SimPO aligns its reward function with the LLM’s generation process. Let π_θ denote our model. Given input problem q , the length-normalized probability of the model generating an answer a is:

$$p_\theta(a | q) = \frac{1}{|a|} \log \pi_\theta(a | q) = \frac{1}{|a|} \sum_{i=1}^{|a|} \log \pi_\theta(a_i | q, a_{<i}) \quad (2)$$


```

Bug localization problem  $q$ :
# You are a hardware engineer proficient in Verilog. Below I give you a
hardware design description and the corresponding code implementation
with the buggy line. Your task is to identify this buggy line.
# Hint: the buggy line may not conform to the design description, or has
logic/functional error.
# Requirements: You should only output the SPECIFIC buggy line identified.
# Design description: {Design description}
# Code implementation: {Buggy code}

Correct answer  $a_w$ :
# always @ (negedge start) begin // The actual buggy line

Incorrect answer  $a_l$ :
# if (cnt == 3) // The non-buggy line that is most frequently misidentified as
buggy by the model

```

Fig. 5. A training sample for reinforcement learning.

where $\log \pi_\theta(a_i | q, a_{<i})$ ¹ denotes the probability of generating the i -th token a_i given the input problem q and the first $i-1$ tokens of a .

For input problem q and answer a , SimPO defines the reward as $\beta \times p_\theta(y | x)$, where β is a constant that controls the reward gap between different answers.

SimPO then applies the Bradley-Terry objective function [28] to compute the difference between the rewards of a_w and a_l . To enhance a_w 's advantage over a_l , SimPO adds a difference term γ to ensure a_w 's reward exceeds a_l 's by at least γ . The resulting objective function is:

$$\mathcal{L}(\pi_\theta) = \mathbb{E} [\log \sigma(\beta \times (p_\theta(a_w | q) - p_\theta(a_l | q)) - \gamma)] \quad (3)$$

SimPO directly employs generation probabilities to define reward and objective functions. By maximizing the objective function, the model is more likely to produce the preferred output a_w instead of a_l . Since a_l is the non-buggy line that the model tends to misidentify, this approach reduces common localization errors.

III. EVALUATION

A. Experimental Settings

1) Training Setups

We train LiK using 8 A800-80G GPUs. We use cosine annealing to adjust learning rate during the training process, with 10% of total steps allocated for warmup. The continual pre-training, supervised fine-tuning, and reinforcement learning take approximately 1, 3, and 20 hours respectively.

2) Evaluation Metrics

We apply pass@ k , a standard metric for evaluating language models' performance on coding-related tasks [29]. It measures the probability of finding at least one correct answer among k repeated generations from models:

$$\text{pass}@k := \mathbb{E} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right] \quad (4)$$

where n denotes the number of generated outputs for each test case. Following prior work on LLM-based Verilog coding [30]–[33], we set n to 20 to obtain reliable performance estimates. We evaluate both pass@1 and pass@5 metrics, where pass@1 measures the model's accuracy with single generations, and pass@5 assesses the performance when generating multiple solutions.

¹This value is generally calculated through forward propagation in the transformer model. For details, please refer to [27].

3) Baselines

We compare LiK with both LLM-based and non-LLM-based Verilog functional bug localization techniques:

- **LLM baselines.** Since no prior work has applied LLMs to Verilog functional bug localization, we establish two baseline categories: (a) Commercial LLMs, including GPT-series (GPT-3.5, GPT-4 and GPT-o1-preview) and Claude-3.5-Sonnet; (b) LLMs specially designed for tasks related to Verilog coding, including Thakur et al. [34], RTLCode [30] and VeriSeek [33]. In our experiments, although we explicitly instruct the model to pinpoint the specific buggy line, these LLMs sometimes respond with complete code blocks, and their output formats vary significantly across different responses. Given this inconsistency, we use a relaxed evaluation criterion: we consider a response to be correct as long as the actual buggy line appears in the response. This yields more optimistic results than the actual performance.
- **The traditional non-LLM method.** We evaluate Strider [9], which is the SOTA Verilog functional bug localization method. Strider identifies a list of possible buggy lines by comparing the waveform outputs between the buggy and golden implementations. Following the same evaluation method as LLM baselines, we consider Strider's output to be correct if its output list includes the actual buggy line.

For LiK, we apply a stricter evaluation criterion: its output must exactly match the actual buggy line. We mark the output as incorrect if it includes any additional lines.

4) Test Dataset

Currently, there is no benchmark for Verilog functional bug localization using LLMs. Test datasets employed in existing works for Verilog bug localization do not have code descriptions, making them unsuitable for our problem where natural language understanding is essential. Therefore, we create a test dataset based on RTLLM [35], a benchmark widely used for evaluating LLMs on Verilog coding tasks. RTLLM consists of 29 typical hardware module designs, where each design includes:

- A design description crafted by experienced hardware engineers. It specifies only the functional requirements and interfaces, without revealing any internal implementation details.
- A golden Verilog implementation (35-175 lines of code).
- A testbench that includes input stimuli and expected outputs.

To create our test dataset, we systematically introduce bugs into the golden code according to the common Verilog error types discussed in Sec. II-A2. Each buggy code maintains correct syntax but fails the testbench verification. The final dataset contains 102 test cases, with bug types distributed as follows:

Bug type	Operators	Numerical value	Variable name	Keyword	Edge
Number	25	26	24	13	14

B. Experimental Results

1) Localization Accuracy

Our experimental results demonstrate the effectiveness of LiK in locating functional bugs in Verilog code. As shown in Table I, the base model exhibits limited performance. This is because the base model's training corpus lacks sufficient data related to Verilog programming and bug localization. Through our training on specialized datasets, LiK extends the base model and demonstrates strong capability in locating functional bugs in Verilog code.

TABLE I
RESULTS ON LOCALIZATION ACCURACY.

Type	Method	pass@1(%)	pass@5(%)
LLM	Thakur et al.,	10.44	20.29
	RTLCoder	17.55	19.12
	Veriseek	16.32	23.84
	GPT-3.5	40.25	50.87
	GPT-4	77.64	85.86
	GPT-o1-preview	78.08	91.83
non-LLM	Claude-3.5	90.8	93.11
	Strider	58.8	58.8
Base Model	Deepseek-Coder-V2-Lite-16B	13.43	19.59
Ours	LiK	93.33	94.10

Table I shows that LiK outperforms all LLM baselines in our experiments. Note that as mentioned in Sec. III-A3, the reported performance of LLM baselines is higher than the actual, as their responses may coincidentally include the buggy line rather than identifying it through accurate localization. Additionally, Verilog-specific LLMs demonstrate relatively poor performance. This is because they are primarily trained for code generation tasks rather than bug localization. Their training datasets mainly consist of hardware design specifications and their corresponding Verilog implementations, with few content related to bug localization.

LiK demonstrates superior localization accuracy compared to Strider. Additionally, we observe that Strider’s output contains 6.67 lines for each test case on average. This multi-line output is inherent to its methodology: it first identifies mismatched output signals, then traces through the code’s abstract syntax tree to find all statements that might affect these signals. Furthermore, Strider requires expert-written testbenches to generate waveform files for bug localization. In comparison, LiK directly locates bugs without the need for verification tools like testbenches, reference models, or other EDA tools. This makes LiK more efficient for bug localization.

2) Ablation Studies

Table II demonstrates LiK’s performance improvements across different training phases. Here, $\text{LiK}_{PT_{1-w/oC}}$ represents the model pre-trained using only Verilog corpus without C corpus and reasoning thoughts. LiK_{PT_1} and $\text{LiK}_{PT_{12}}$ denote the model after the first and second round of continual pre-training, and $\text{LiK}_{PT_{12}+SFT}$ and $\text{LiK}_{PT_{12}+SFT+RL}$ represent the model after supervised fine-tuning and reinforcement learning, respectively.

TABLE II
ABLATION STUDY OF LiK’S TRAINING STAGES

Model	pass@1(%)	pass@5(%)
$\text{LiK}_{PT_{1-w/oC}}$	16.63	24.31
LiK_{PT_1}	18.97	27.01
$\text{LiK}_{PT_{12}}$	29.15	38.76
$\text{LiK}_{PT_{12}+SFT}$	87.54	90.83
$\text{LiK}_{PT_{12}+SFT+RL}$	93.33	94.10

LiK pre-trained with only Verilog corpus achieves a modest performance. Adding C corpus brings slight improvements, with pass@1 reaching 18.97%. The incorporation of bug localization reasoning thoughts further enhances the model’s performance to 29.15% pass@1, which indicates enhanced understanding of bug localization problems. However, these performances remain relatively low. This weakness stems from the unsupervised nature of continual pre-training, during which the model has not explicitly learned to locate buggy lines.

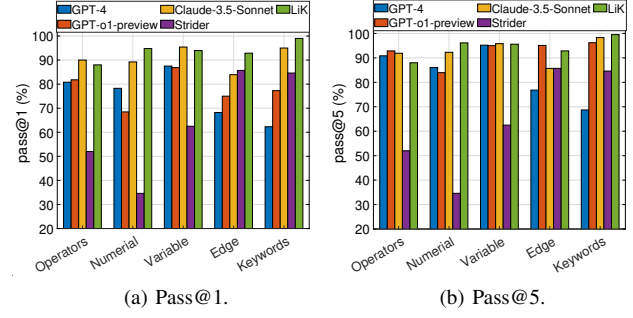


Fig. 6. The performance on different types of bugs.

Supervised fine-tuning leads to a substantial performance increase, with pass@1 accuracy reaching 87.54%. This improvement is attributed to the supervised nature of this training phase, which directly aligns the model’s outputs with bug localization objectives.

The reinforcement learning phase further improves the model’s performance, pushing the pass@1 metric to 93.33%. This phase helps the model avoid common errors and prioritize correct bug localization. We further discuss its impact in Sec. IV-A.

3) Results with Different Bug Types

This part examines the performance in locating different types of bugs. As shown in Fig. 6(a) and 6(b), LiK demonstrates superior performance compared to all baselines in locating bugs related to numerical values, keywords, and edge conditions. For bugs involving variable names and operators, Claude-3.5-Sonnet achieves better results than LiK. Compared to GPT series, LiK shows higher pass@1 but slightly lower pass@5 in these categories.

The performance variation can be explained by the inherent characteristics of different bug types. Edge-related and keyword-related bugs usually directly conflict with design descriptions. On the other hand, bugs related to numerical values, operators, and variable names are often embedded in the internal implementation logic, thus requiring more complex reasoning to be localized.

Two main factors limit LiK’s ability in complex reasoning tasks: its smaller model size and insufficient training data. Despite our best efforts to collect data, due to the closed nature of the hardware design community, we are only able to obtain ~300MB of training data. Future work will focus on expanding the training dataset to improve model performance.

Notably, all LLMs including ours consistently outperform Strider, the SOTA non-LLM method, across all bug categories. This demonstrates the effectiveness of LLMs in Verilog bug localization.

4) Can LiK benefit LLM-based Verilog debugging?

With the rapid development of LLMs, leveraging them for automated Verilog debugging has emerged as a promising direction [15], [31], [36], [37]. The ultimate goal of bug localization is to facilitate the automated debugging processes. To demonstrate how LiK can benefit LLM-based Verilog debugging systems, we conducted experiments combining LiK’s results with three baseline debugging methods using LLMs: GPT-3.5-based, GPT-4-based, and the SOTA LLM-based Verilog debugging system MEIC [15].

For two GPT-based methods, we input the design description and buggy code, and ask GPT to fix functional bugs. MEIC implements an end-to-end debugging framework with iterative cycles, during which two fine-tuned LLMs work in tandem: one repairs the buggy code based on compiler/simulator feedback, while the other scores the repairs to guide subsequent iterations. This process runs until the code passes testbench verification or reaches 10 iterations.

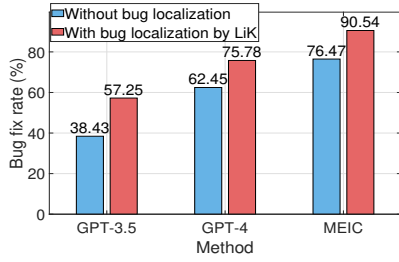


Fig. 7. The success rate of bug fixing after incorporating the result of LiK.

To assess the impact of LiK, we add two elements at the end of the LLM prompts for baseline methods: the buggy lines identified by LiK, and a clear statement indicating these are buggy lines. For MEIC, we add this information to the prompt of the LLM responsible for code repairing. We conduct 20 repeated experiments on our test dataset and measure the bug-fixing success rate. A bug fix is considered successful only when the repaired code passes all the test cases in its testbench.

As shown in Fig. 7, the integration of LiK's bug localization significantly improves the performance of all baselines. Notably, MEIC's bug-fixing success rate is increased by 14%. Also, we observe that LiK+MEIC achieves 99.35% success rate on the test cases that MEIC could handle alone. These results highlight LiK's potential in enhancing end-to-end automatic Verilog debugging systems.

IV. DISCUSSIONS

A. The Impact of Reinforcement Learning

To further analyze the impact of the reinforcement learning phase, we examine LiK's performance before and after it across different temperature values. Temperature is a parameter ranging from 0 to 1 that controls the randomness of model's output, in which higher temperature leads to more diverse outputs.

As shown in Fig. 8, before reinforcement learning, LiK's pass@5 varies significantly across temperatures and is much higher than its pass@1. This gap indicates the output instability for identical test cases, leading to errors in bug localization. After reinforcement learning, the gap between pass@5 and pass@1 becomes smaller, and pass@5 becomes more stable with changes in temperature. Moreover, both pass@1 and pass@5 increase across all temperature settings. This shows that by utilizing incorrect bug localization as negative examples, reinforcement learning reduces errors caused by output instability, thereby improving bug localization accuracy.

B. Why not Using Reasoning Thoughts for Supervised Fine-tuning?

Previous research [24] shows that prompting LLMs to generate reasoning thoughts rather than directly providing the answers can improve accuracy. This suggests that LiK might benefit from generating reasoning thoughts for bug localization. For this to work, during the supervised fine-tuning, we need to use reasoning thoughts as expected outputs, rather than just the buggy line.

To test this hypothesis, we compare the performance of two models: one trained with bug localization reasoning thoughts and another with only buggy lines (our current approach). We generate reasoning thoughts for bug localization using the method and seed dataset in Sec. II-A2, and use these thoughts as training data for the reasoning-based model. We evaluate both models on the test set used in Sec. III. Results show that the model trained with reasoning thoughts achieves pass@1 and pass@5 scores of 73.48% and 77.69%, while our current model trained with only buggy lines performs better, reaching 87.54% and 90.83%.

To understand this performance gap, we analyze the training loss of both models. As shown in Fig. 9, the model trained with

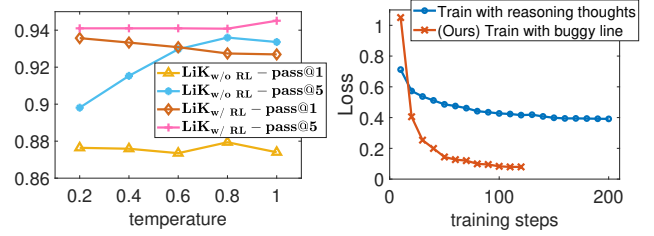


Fig. 8. Performance vs temperature. Fig. 9. The training loss with different types of data.

reasoning thoughts has higher training loss. This suggests that reasoning thoughts introduce additional noise into the training process. Intuitively, reasoning thoughts are not our target output - they are intermediate steps towards the bug localization. The extra complexity from reasoning thoughts makes it harder for the model to focus on the main task of bug localization. Therefore, we use only buggy lines as the expected output during supervised fine-tuning.

V. RELATED WORKS

Several non-LLM approaches [1]–[9] have been proposed for Verilog bug localization. They typically provide a set of potential buggy lines rather than pinpointing the exact bug location. Moreover, they rely on verification tools that require expertise to develop.

No prior research has explored the use of LLMs for Verilog bug localization. Other related works can be categorized into two groups:

(1) The first category [16], [30], [33], [34], [38] proposes LLMs specifically for Verilog. However, their primary focus is on code generation, with their training data mainly consisting of design documents and corresponding implementations, containing little content on Verilog bug localization. Therefore, they are not suitable for the bug localization problem.

(2) The second category [15], [31], [36], [37] develops LLM-based frameworks for Verilog debugging, which includes several LLMs as agents and each agent completes one intermediate step. However, none of these frameworks includes bug localization. Our experiments demonstrate that integrating LiK into the SOTA LLM-based functional bug repairing framework MEIC [15] significantly improves its performance. This highlights the importance of bug localization in LLM-based debugging workflows.

VI. CONCLUSION

This paper proposes LiK, an LLM for locating the functional bugs in Verilog code. LiK is trained through two rounds of continuous pre-training, supervised fine-tuning, and reinforcement learning. Experimental results show that LiK achieves high localization accuracy. By integrating LiK's results, the performance of the SOTA LLM-based Verilog debugging system is significantly improved. This shows LiK's potential to support auto-verification tools for Verilog debugging.

As for the future work, we plan to enhance LiK by leveraging additional expert knowledge and EDA tool feedback, and generating more training data specialized for bug localization. We will also extend LiK for more complex hardware designs and other hardware description languages such as VHDL, SystemVerilog, and Chisel.

ACKNOWLEDGEMENT

We would like to thank the anonymous reviewers for their helpful feedback. This work is supported by National Key Research and Development Program (Grant No. 2024YFB4405600), the National Natural Science Foundation of China (Grant No. 62472086), the Basic Research Program of Jiangsu (Grants No. BK20243042, BG2024010), the Start-up Research Fund of Southeast University (Grant No. RF1028624005), the National Natural Science Foundation of China (Grants No. U22A2025 and 92464301).

REFERENCES

- [1] J. Wu, Z. Zhang, D. Yang, X. Meng, J. He, X. Mao, and Y. Lei, "Fault localization for hardware design code with time-aware program spectrum," in *IEEE ICCD 2022*, pp. 537–544.
- [2] Z. Zhang, Y. Li, L. Xia, J. Xue, J. Wu, and X. Mao, "An effective fault localization approach for verilog based on enhanced contexts," *Frontiers of Computer Science*, vol. 18, 06 2024.
- [3] D. Pal and S. Vasudevan, "Symptomatic bug localization for functional debug of hardware designs," in *IEEE VLSID 2016*, pp. 517–522.
- [4] B. Kumar, V. S. Vineesh, P. Nemade, and M. Fujita, "Aries: A semi-formal technique for fine-grained bug localization in hardware designs," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 12, pp. 5709–5721, 2022.
- [5] S. Srinivasan and R. Vemuri, "Model checking leveraged error localization for complex rtl designs," in *IEEE ICCD 2022*, pp. 585–592.
- [6] P. Santiesteban, Y. Huang, W. Weimer, and H. Ahmad, "Cirfix: Automated hardware repair and its real-world applications," *IEEE Transactions on Software Engineering*, vol. 49, no. 7, pp. 3736–3752, 2023.
- [7] J. Wu, Z. Zhang, D. Yang, J. Xu, J. He, and X. Mao, "Knowledge-augmented mutation-based bug localization for hardware design code," *ACM Trans. Archit. Code Optim.*, apr 2024, just Accepted.
- [8] G. Stracquadanio, S. Medya, S. Quer, and D. Pal, "Veribug: An attention-based framework for bug localization in hardware designs," in *IEEE/ACM DATE 2024*, pp. 1–2.
- [9] D. Yang, J. He, X. Mao, T. Li, Y. Lei, X. Yi, and J. Wu, "Strider: Signal value transition-guided defect repair for hdl programming assignments," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 5, pp. 1594–1607, 2024.
- [10] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, "Software testing with large language models: Survey, landscape, and vision," *IEEE Transactions on Software Engineering*, vol. 50, no. 4, pp. 911–936, 2024.
- [11] H. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt, "Examining zero-shot vulnerability repair with large language models," in *IEEE S&P 2023*, pp. 2339–2356.
- [12] S. B. Hossain, N. Jiang, Q. Zhou, X. Li, W.-H. Chiang, Y. Lyu, H. Nguyen, and O. Tripp, "A deep dive into large language models for automated bug localization and repair," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, 2024.
- [13] DeepSeek-AI, Q. Zhu, and D. G. et al., "Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence," 2024. [Online]. Available: <https://arxiv.org/abs/2406.11931>
- [14] A. J. et al., Yuntao Bai, "Training a helpful and harmless assistant with reinforcement learning from human feedback," 2022. [Online]. Available: <https://arxiv.org/abs/2204.05862>
- [15] K. Xu, J. Sun, Y. Hu, X. Fang, W. Shan, X. Wang, and Z. Jiang, "Meic: Re-thinking rtl debug automation using llms," in *IEEE ICCAD 2024*, to appear. [Online]. Available: <https://arxiv.org/abs/2405.06840>
- [16] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg, "Verigen: A large language model for verilog code generation," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 29, no. 3, 2024.
- [17] "Opencores." [Online]. Available: <https://opencores.org>
- [18] E. J. Hu, yelong shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *ICLR 2022*. [Online]. Available: <https://openreview.net/forum?id=nZeVKeeFYf9>
- [19] Z. Pei, H. Zhen, M. Yuan, Y. Huang, and B. Yu, "BetterV: Controlled verilog generation with discriminative guidance," in *ACM ICML 2024*, vol. 235, pp. 40 145–40 153.
- [20] K. Gupta, B. Thérien, A. Ibrahim, M. L. Richter, Q. Anthony, E. Belilovsky, I. Rish, and T. Lesort, "Continual pre-training of large language models: How to (re)warm your model?" 2023. [Online]. Available: <https://arxiv.org/abs/2308.04014>
- [21] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "Codesearchnet challenge: Evaluating the state of semantic code search," 2020. [Online]. Available: <https://arxiv.org/abs/1909.09436>
- [22] A. Broder, "On the resemblance and containment of documents," in *Proceedings. Compression and Complexity of SEQUENCES 1997 (Cat. No.97TB100171)*, 1997, pp. 21–29.
- [23] O. Jafari, P. Maurya, P. Nagarkar, K. M. Islam, and C. Crushev, "A survey on locality sensitive hashing algorithms and their applications," 2021. [Online]. Available: <https://arxiv.org/abs/2102.08942>
- [24] J. Wei, X. Wang, D. Schuurmans, M. Bosma, b. ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *NeurIPS 2022*, vol. 35, pp. 24 824–24 837.
- [25] S. Sudakrishnan, J. Madhavan, E. J. Whitehead, and J. Renau, "Understanding bug fix patterns in verilog," in *IEEE/ACM MSR 2008*, p. 39–42.
- [26] Y. Meng, M. Xia, and D. Chen, "Simpot: Simple preference optimization with a reference-free reward," 2024. [Online]. Available: <https://arxiv.org/abs/2405.14734>
- [27] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2023. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [28] "Bradley-terry model." [Online]. Available: https://en.wikipedia.org/wiki/Bradley%E2%80%93Terry_model
- [29] M. Chen and J. T. et al., "Evaluating large language models trained on code," 2021. [Online]. Available: <https://arxiv.org/abs/2107.03374>
- [30] S. Liu, W. Fang, Y. Lu, J. Wang, Q. Zhang, H. Zhang, and Z. Xie, "Rtl-coder: Fully open-source and efficient llm-assisted rtl code generation technique," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, pp. 1–1, 2024.
- [31] Y.-D. Tsai, M. Liu, and H. Ren, "Rtlfixer: Automatically fixing rtl syntax errors with large language models," 2024. [Online]. Available: <https://arxiv.org/abs/2311.16543>
- [32] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "Invited paper: Verilog-eval: Evaluating large language models for verilog code generation," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2023, pp. 1–8.
- [33] W. Ning, Y. Bingkun, Z. Jie, W. Xi, J. Zhe, and G. Nan, "Large language model for verilog generation with golden code feedback," 2024. [Online]. Available: <https://arxiv.org/abs/2407.18271>
- [34] S. Thakur, B. Ahmad, Z. Fan, H. Pearce, B. Tan, R. Karri, B. Dolan-Gavitt, and S. Garg, "Benchmarking large language models for automated verilog rtl code generation," in *IEEE/ACM DATE 2023*, pp. 1–6.
- [35] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "Rtlml: An open-source benchmark for design rtl generation with large language model," in *IEEE/ACM ASP-DAC 2024*, p. 722–727.
- [36] H. Huang, Z. Lin, Z. Wang, X. Chen, K. Ding, and J. Zhao, "Towards llm-powered verilog rtl assistant: Self-verification and self-correction," 2024. [Online]. Available: <https://arxiv.org/abs/2406.00115>
- [37] X. Yao, H. Li, T. H. Chan, W. Xiao, M. Yuan, Y. Huang, L. Chen, and B. Yu, "Hdldebugger: Streamlining hdl debugging with large language models," 2024. [Online]. Available: <https://arxiv.org/abs/2403.11671>
- [38] W. Fu, S. Li, Y. Zhao, H. Ma, R. Dutta, X. Zhang, K. Yang, Y. Jin, and X. Guo, "Hardware phi-1.5b: A large language model encodes hardware domain specific knowledge," in *IEEE/ACM ASP-DAC 2024*, pp. 349–354.